Machine Learning: Simple Neural Networks

David Cornu

M2-OSAE 2022

Lessons materials

Slides, exercises, codes, corrections and datasets are available on GitHub and will be updated regularly:

http://github.com/Deyht/ML_OSAE_M2

Useful commands:

git clone https://github.com/Deyht/Deyht/ML_OSAE_M2

git pull

To avoid loosing your work on forced pull update, copy all files from the clone repository into a working directory!

The brain as model

« There is a fantastic existence proof that learning is possible, which is the bag of water and electricity (together with a few trace chemicals) sitting between your ears […] wich is the squishy thing that your skull protects »

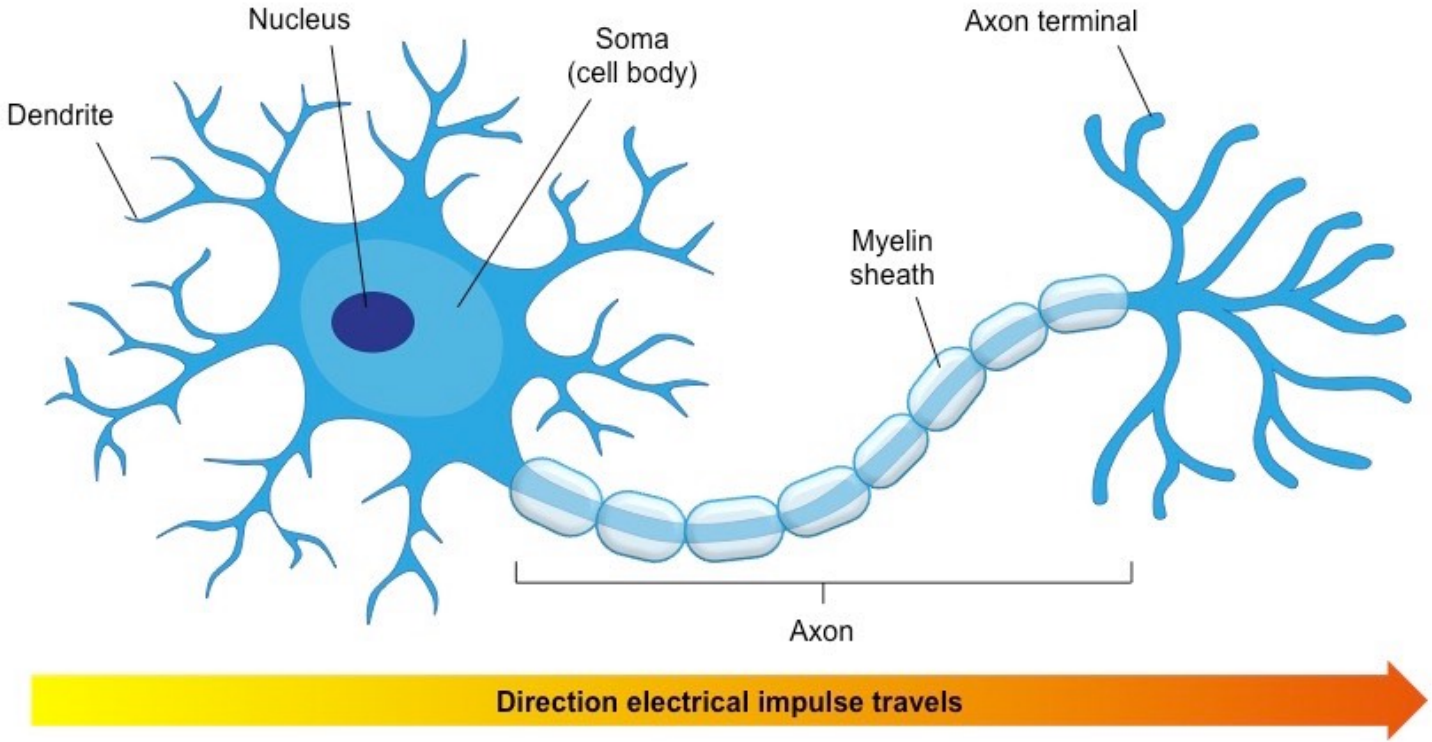
**Stephen Marsland*

The brain does exactly what we want for data analysis:

- Extract complex information in a compressed form
- Deal with noisy and/or inconsistent data
- Work in high dimensional spaces
- Give the appropriate answer most of the time
- Provide results very quickly
- Remain robust through aging (neurons loss)

The biological neuron

Elementary brick of a biological brain (10^{11} in the human brain).
The idea will be to use it as a model to emulate learning capabilities.



Perform the sum of various electrochemical input signals. If this sum is enough it will send a new signal through its axon to transfer information (most likely toward other neurons).

Biological Neural Networks

A connection between two neurons is called a synapse (10^{14} in the human brain). We can consider that, a neuron = a simple binary compute unit that either “Fire” or “not-fire” in response to a signal.

➡ **In this view, a brain is a massively parallel super-computer of 10^{11} processing units**

A simplified view of how it learns

The synapses represents the “strength” of the connection between two neurons.

Learning = modifying this connection (either positive or negative and change its intensity)

➡ This is called **plasticity**

The Hebb law define a simplified rule for learning :

If two neurons “fire” at the same time, there must be some **correlation** between them and their connection must be strengthen.

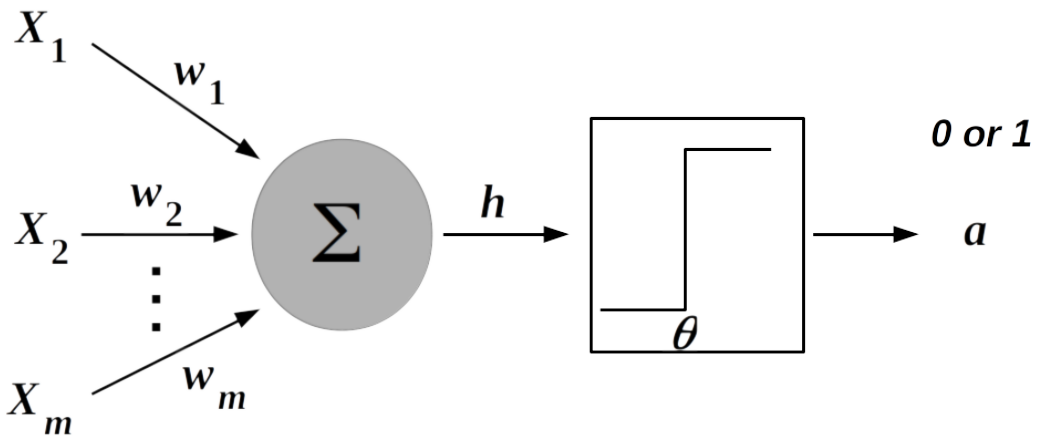
➡ This is called **conditioning**

**These rules by themselves are not enough to train a neural system, but are a good illustration of the type of processes that might take place in the biological brain.*

In order to create an algorithm out of these biological concepts, one first need to create a mathematical model.

Model of a Neuron

The following is based on the mathematical model from McCulloch and Pitts that was inspired by the biological neuron.

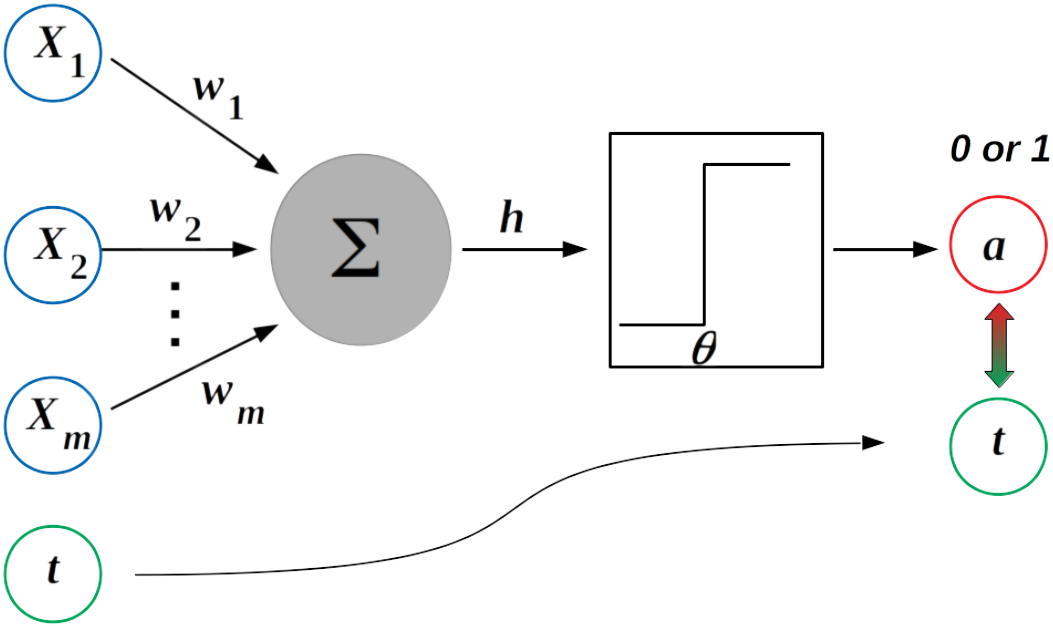


Its main components are :

- An input vector X_i that represents the dimensions of a given object
- A set of weights w_i that links the various input dimensions to the neuron
- A sum function h that defines how these weights are combined with the input dimensions
- An activation function $g(h)$ that defines if the network should remain in a "0" state or should be activated to a "1" state, depending on the results of the previous sum.

$$h = \sum_{i=1}^{m} X_i \omega_i \qquad a = g(h) = \begin{cases} 1 & \text{if} \quad h > \theta \\ 0 & \text{if} \quad h \leq \theta \end{cases}$$

Training a Neuron



$$h = \sum_{i=1}^{m} X_i \omega_i \qquad a = g(h) = \begin{cases} 1 & \text{if} \quad h > \theta \\ 0 & \text{if} \quad h \leq \theta \end{cases}$$

The learning process for this model is supervised
→ Each input vector is associated to a target value

So what is the purpose of the neuron if the expected value is already known ?

➡ **Generalization**

Find patterns in the data distribution in the parameter space so it can perform prediction on vectors with unseen (but close) values.

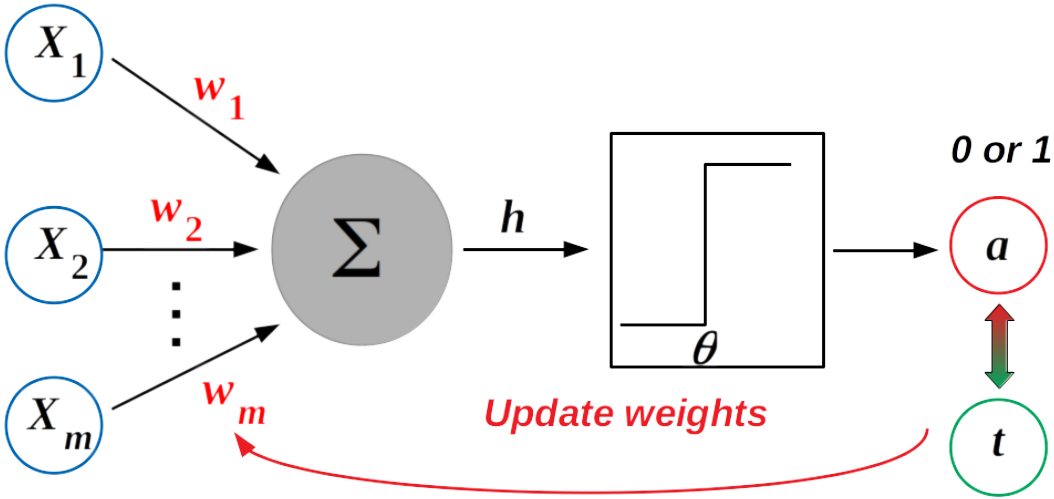
In practice such a simple neuron optimize a linear separation in the parameter space (**hyperplane**).

How does this model learn ?

Which parameters can or cannot be modified ?

The input, output and targets are fixed, so the learning relies on modifying the **weights and the activation threshold**.

Training a Neuron



$$\omega_i \leftarrow \omega_i - \eta\ (a - t)\ X_i$$

$$h = \sum_{i=1}^{m} X_i \omega_i \qquad a = g(h) = \begin{cases} 1 & \text{if} \quad h > \theta \\ 0 & \text{if} \quad h \leq \theta \end{cases}$$

How to proceed when the output does not correspond to the target ?

There are *m* weights w_i associated with the network corresponding to the input dimensions.

How to modify the weights ?

- If the output state is 1 while it should be 0, the weights need to be lowered
- If the output state is 0 while it should be 1, the weights need to be increased

To quantify this modification one need to choose an error function *E.*

$$E = 0.5 \times (a - t)^2$$

In the end, the weight update is defined as proportional to the input value, to the derivative of the error function for each dimension, and scaled by a learning rate* factor.

$$\omega_i \leftarrow \omega_i - \eta\ (a - t)\ X_i$$

**Typically between 0.1 and 0.4, details will be given later.*

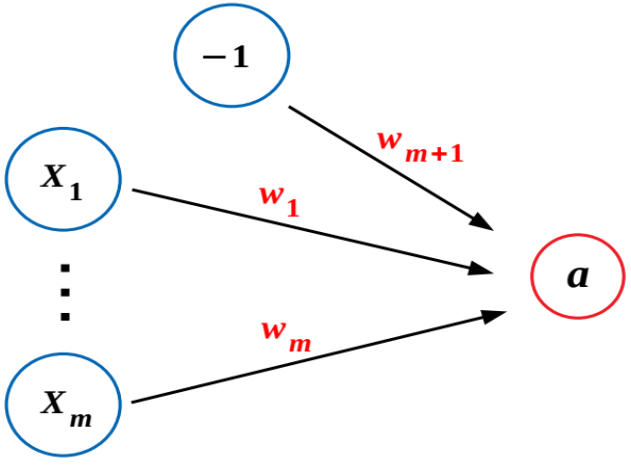
The bias node

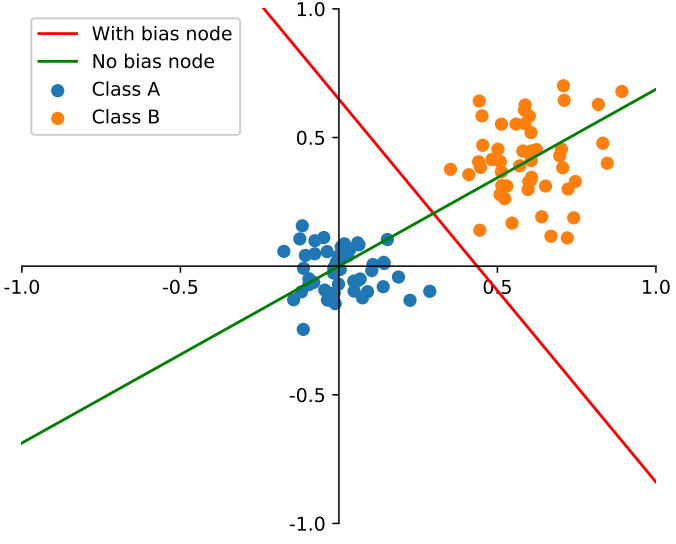
Problem with the previous formalism

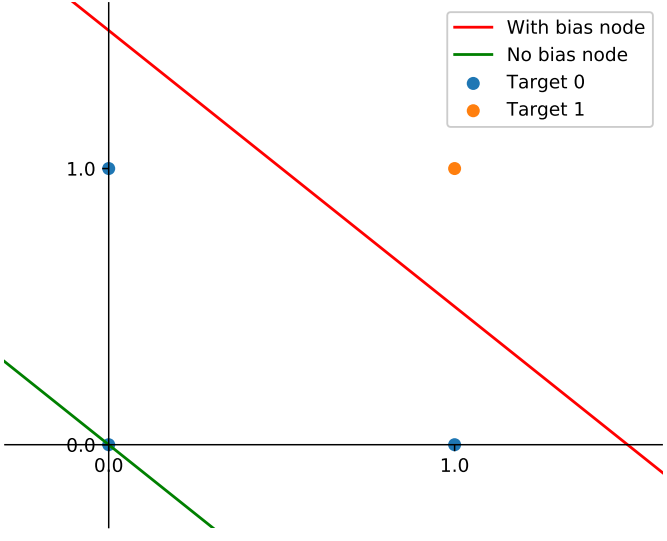
The linear separation as a fixed *f(0) = 0 point*.
The weight correction is also 0 regardless of the input value.

Solution

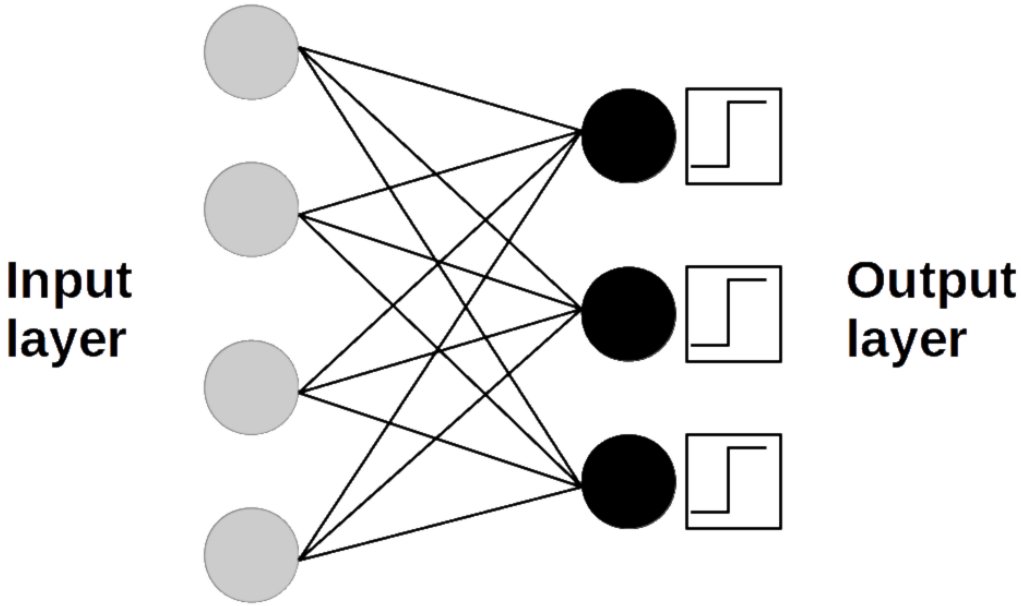
Add a bias input node that act as a supplementary input with a constant value (most of the time -1). It still has its **own variable weight** in order to learn the shift of the intercept position. The size of both the input vector and the weights vector is now *m+1*.







The Perceptron algorithm



As we saw previously, a single neuron only provide a linear separation, which is insufficient for many applications.

The simplest way to combine neurons on a single problem is to **stack them independently**. Each neuron is connected to the input vector with its own set of weights.

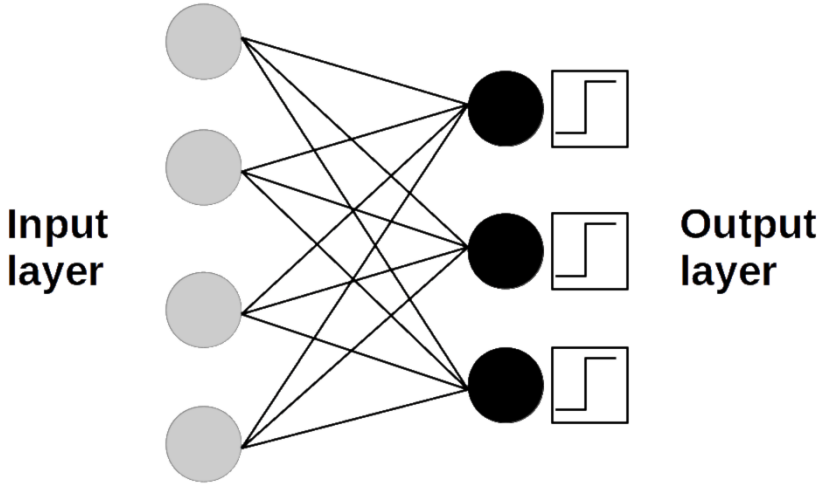
The training procedure is identical, but this time there is an index j to represent the neurons, and the weights are now in the form of a 2D matrix.

$$h_j = \sum_{i=1}^{m+1} X_i \omega_{ij} \qquad a_j = g(h_j) = \begin{cases} 1 & \text{if} \quad h_j > \theta \\ 0 & \text{if} \quad h_j \leq \theta \end{cases}$$

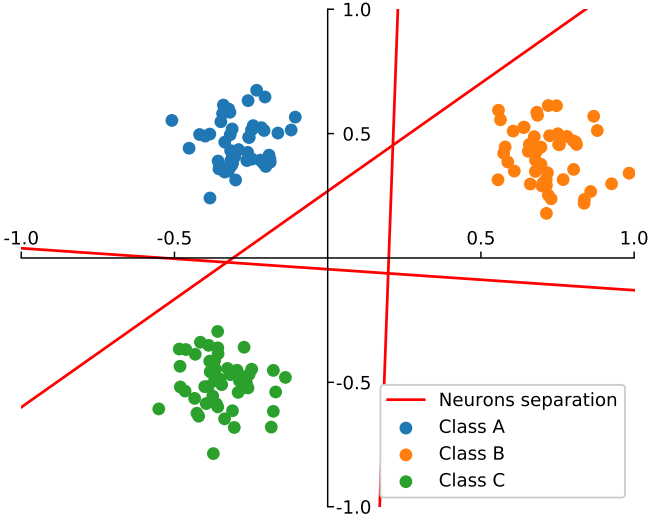
$$\omega_{ij} \leftarrow \omega_{ij} - \eta \left(a_j - t_j \right) \times X_i$$

This architecture, and the associated training procedure is called the **Perceptron** (Rosenblatt 1958).

The Perceptron algorithm



C1	C2	C3
1	0	0
0	1	0
0	0	1



How to have multiple neurons working on the same problem ?

1) Encode the output vector (e.g in binary format)
2) Have on neuron per output class and target a format with a 1 only in the appropriate class, e.g [1,0,0]-[0,1,0]-[0,0,1]

**Warning each class must be linearly separable from all the others, since each neuron is only performing a linear separation in this context.*

The Perceptron algorithm

- **Initialization**
  - Set the starting weights to small random values (positive and negative). Can be drawn from a uniform or Gaussian distribution centered on zero.

- **Training**
  - For a given number of steps T, or until the output is “correct”
    - For each input vector
      - Compute the activation of each neuron *j* using the activation function *g :*

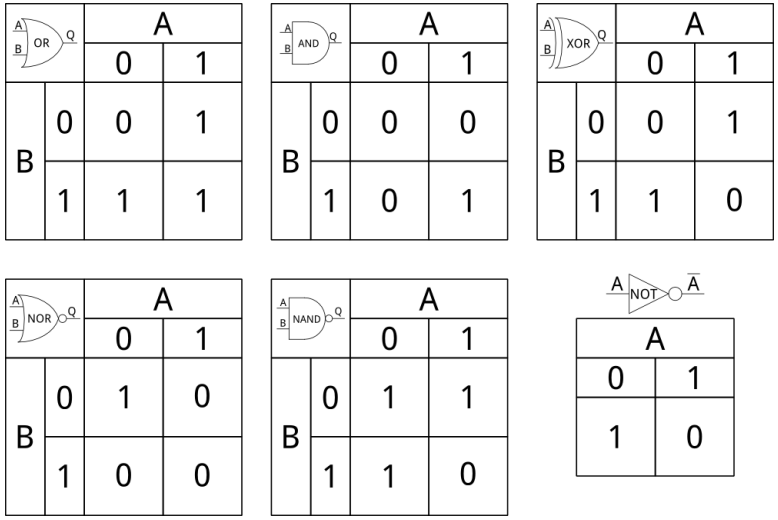
$$y_j = g\left(\sum_{i=0}^{m} w_{ij} x_i\right) = \begin{cases} 1 & \text{if } \sum_{i=0}^{m} w_{ij} x_i > 0 \\ 0 & \text{if } \sum_{i=0}^{m} w_{ij} x_i \leq 0 \end{cases} \tag{1}$$

      - Update each weight individually using :

$$w_{ij} \leftarrow w_{ij} - \eta \left(y_j - t_j\right) \cdot x_i \tag{2}$$

- **Inference**
  - Compute the final activation of each neuron *j* for each input vector to test

First application : logical gates



OR		A	
		0	1
B	0	0	1
	1	1	1

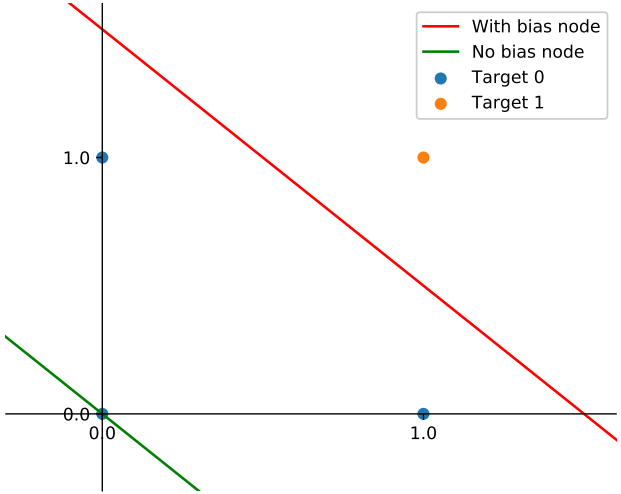
AND		A	
		0	1
B	0	0	0
	1	0	1

XOR		A	
		0	1
B	0	0	1
	1	1	0

NOR		A	
		0	1
B	0	1	0
	1	0	0

NAND		A	
		0	1
B	0	1	1
	1	1	0

NOT	
A	1
0	1
1	0



A very simple application of this algorithm, is to make it learn a logical door, here the OR or AND gate.

For this simple application the Perceptron has a two dimensional input vector with 4 possibilities.

The output is made of a single binary neuron to predict either the 0 or 1 state of the gate.

As a first exercise write a simple program that declare an array with all the possible inputs and the associated targets. Identify all the necessary variables, initialize the weights with small values, and try to train this neuron over a few iterations following the Perceptron algorithm.

Do not forget the bias node !

Display the output at each iteration to watch the network converge toward a stable solution.

Why is the **XOR gate** problematic with this algorithm ?

How to solve the problem ?

The Pima Indian dataset

Dataset from the UCI Machine Learning repository.

Provide 8 physiological measurements for a population of 768 female native Americans, and provide a class depending of the individual having developed diabetes or not.

The objective of this exercise is to implement a Perceptron to predict if a person has diabetes based solely on the 8 input measurements.

Despite its limitations, a simple Perceptron can reach up to 70% accuracy on this dataset.

Modify your previous program to read the Pima dataset, adapt the input dimension (including bias), and the corresponding computations.

Find a method to compute the "accuracy" and its evolution during the training phase. Question the limits of your accuracy definition, and of a way to ensure that you are testing the generalizing capabilities of the method.

Attribute Number	Attribute
1	Patient age
2	Body mass index (kg/m2)
3	Concentration of plasma glucose
4	2-h serum insulin (mu U/mL)
5	Thickness of triceps skin-fold (mm)
6	Pedigree function of diabetes
7	Number of times patient pregnant
8	Diastolic blood pressure (mmHg)
9	Class 0 or 1

Input dimension : 8 (+1)

Output dimension : 1
(or 2 if using one neuron per class)

Number of example : 768

Class distribution : 500 class 0; and 268 class 1

The Iris dataset

Dataset from the UCI Machine Learning repository.

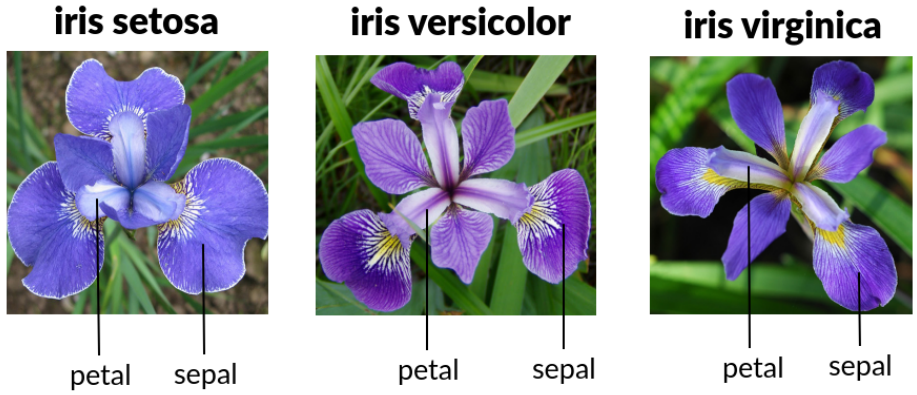
Provide 4 size measurements for a set of 150 Iris flowers, and provide a membership to one of 3 predefined classes.

The objective of this exercise is to implement a Perceptron to predict the corresponding class based on the 4 input measurements for each flower.

Despite its limitations, a simple Perceptron can reach up to 80% accuracy on this dataset.

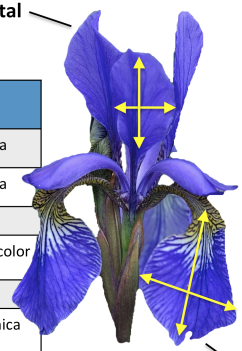
Modify your previous program to read the Iris dataset, adapt the input dimension (including bias), and the corresponding computations.

Find a method to compute the "accuracy" and its evolution during the training phase. Question the limits of your accuracy definition, and think of a way to ensure that you are testing the generalizing capabilities of the method.



Samples
(instances, observations)

	Sepal length	Sepal width	Petal length	Petal width	Class label
1	5.1	3.5	1.4	0.2	Setosa
2	4.9	3.0	1.4	0.2	Setosa
...					
50	6.4	3.5	4.5	1.2	Versicolor
...					
150	5.9	3.0	5.0	1.8	Virginica



Class labels
(targets)

Features
(attributes, measurements, dimensions)

Input dimension : 4 (+1)
Output dimension : 3
Number of example : 150
Class distribution : 50 examples per class